

Q1 Student Management System [BL3 – Apply]

10 marks

Write pseudocode to design a Student class with attributes (name, rollNo, marks) and methods to:

- Accept student details
- Calculate average marks
- Display result (Pass/Fail)

Apply encapsulation and data hiding by restricting direct access to attributes.

OOP Concepts: Encapsulation | Data Hiding | Classes & Objects

Your Answer

```
START
CLASS student
PRIVATE:
    name
    rollno
    mark1,mark2,mark3
    average
METHOD studentDetails(name,rollno,mark1,mark2,mark3)
    this.name<- name
    this.rollno<- rollno
    this.mark1<- mark1
    this.mark2<- mark2
    this.mark3<- mark3
END METHOD
METHOD calculateAverage()
    average<-(mark1+mark2+mark3)/3
END METHOD
METHOD displayResults()
    IF average>=50 THEN
        PRINT "PASS"
    ELSE
        PRINT "FAIL"
    END IF
END METHOD
END CLASS
MAIN
CREATE object s OF student
INPUT name,rollno
INPUT mark1,mark2,mark3
CALL s.studentDetails(name,rollno,mark1,mark2,mark3)
CALL s.calculateAverage()
CALL s.displayResults()
STOP
```

Save Answer

Q3 Employee Salary Calculation [BL3 – Apply]

10 marks

Write pseudocode for an Employee class with encapsulated attributes (empId, name, basicPay, designation). Implement methods to:

Calculate gross salary (basic + HRA + DA)

- Calculate net salary after deductions (PF, tax)

- Display pay slip

Apply data hiding so salary components are not directly accessible.

OOP Concepts: Encapsulation | Data Hiding | Methods

Your Answer

```
START
CLASS Employee
PRIVATE
    empId
    name
    basicPay
    designation
PUBLIC
    METHOD calculateGrossSalary()
        HRA=basicPay*20/100
        DA=basicPay*10/100
        grossSalary=basicPay+HRA+DA
        RETURN grossSalary
    END METHOD
    METHOD calculateNetSalary()
        grossSalary=calculateGrossSalary()
        PF=basicPay*12/100
        TAX=grossSalary*10/100
        netSalary=grossSalary-PF-TAX
        RETURN netSalary
    END METHOD
    METHOD displayPaySlip()
        grossSalary=calculateGrossSalary()
        netSalary=calculateNetSalary()

        DISPLAY"Employee ID:",empId
        DISPLAY"Name:",name
        DISPLAY"Designation:",designation
        DISPLAY"Basicpay:",basicPay
        DISPLAY"Gross Salary:",grossSalary
        DISPLAY"Net Salary:",netSalary
    END METHOD
END CLASS
CREATE Employee object
SET empId
SET name
SET basicPay
SET designation
CALL displayPaySlip()
STOP
```

 Save Answer

Q2 2. Library Book Management [BL3 – Apply]

10 marks

Design a Book class with private attributes (title, author, ISBN, isAvailable). Implement methods to:

- Issue a book (change availability status)
- Return a book
- Display book details using getter methods

Demonstrate object creation for at least 3 books and simulate issue/return operations.

OOP Concepts: Encapsulation | Getters & Setters | Objects

Your Answer

```
START
CLASS Book
PRIVATE:
    title
    author
    ISBN
    isAvailable
Methods:
setDetails(title,author,ISBN)
    Store title,author,ISBN
    isAvailable=TRUE
issueBook()
    IF isAvailable=TRUE THEN
        isAvailable=FALSE
        Print "Book issued"
    ELSE
        print "Book not Available"
    END IF
returnBook()
    isAvailable =TRUE
    print "Book Returned"

displayDetails()
    print title
    print author
    print ISBN
    print Availability Status
END Class
Main program
create book1,book2,book3
set details for all 3 books
call Book1.issueBook()
call Book2.issueBook()
call Book1.returnBook()
call displayDetails() for Book1
call displayDetails() for Book2
call displayDetails() for Book3

STOP
```

Save Answer

Q5 Vehicle Registration System [BL3 – Apply]

10 marks

Design a Vehicle class with attributes (regNo, owner, fuelType). Extend it into:

- TwoWheeler — with engine capacity
- FourWheeler — with seating capacity and AC status

Apply inheritance to reuse common attributes and override a displayDetails() method in each subclass.

OOP Concepts: Inheritance | Method Overriding | Subclass

Your Answer

```
START
CLASS Vehicle
  DECLARE regno
  DECLARE owner
  DECLARE fueltype
METHOD displayDetails()
  DISPLAY regno
  DISPLAY owner
  DISPLAY fueltype
END METHOD
END CLASS
CLASS twowheeler extends vehicle
  DECLARE enginecapacity
  METHOD diplayDetails()
    DISPLAY regno
    DISPLAY owner
    DIAPLAY fueltype
    DISPLAY enginecapacity
  END METHOD
END CLASS
CLASS fourwheeler extends vehicle
  DECLARE seatingcapacity
  DECLARE acstatus
END METHOD
END CLASS
CREATE twowheeler object
ASSIGN registration number,owner,
fuel type and engine capacity
CALL displayDetails()

CREATE fourwheeler object
ASSIGN registration number and AC status
CALL displayDetails()
STOP
```

 Save Answer

Q6 Bank Account Hierarchy [BL4 – Analyze]

10 marks

Develop pseudocode for a BankAccount superclass and two subclasses — SavingsAccount (with interest calculation) and CurrentAccount (with overdraft facility). Analyze and implement:

- Inheritance of common attributes (accNo, balance, holder)
- Method overriding for withdrawal behavior
- How overdraft protection differs from savings limit

Compare the behavior of withdraw() across both subclasses and justify the design choices.

OOP Concepts: Inheritance | Method Overriding | Polymorphism

Your Answer

```
START
CLASS BankAccount
  accno,balance,holder
METHOD withdraw(amount)
  IF amount<=balance
    balance=balance-amount
  ELSE
    PRINT "insufficient balance"
  END IF
END METHOD
END CLASS
CLASS SavingsAccount extends
  bankaccount
METHOD withdraw(amount)
  IF amount<= balance
    balance=balance-amount
  ELSE
    PRINT"savings limit exceeded"
  END IF
END METHOD
END CLASS
CLASS currentaccount extends
  bankaccount
  overdraftlimit
METHOD withdraw(amount)
  IF amount<=balance+overdraftlimit
    balance=balance-amount
  ELSE
    PRINT "overdraft limit exceeded"
  END IF
END METHOD
END CLASS

STOP
```

Save Answer

Q7 Hospital Patient Record System [BL4 – Analyze]

10 marks

Design a Patient base class and analyze how it extends into:

- InPatient — with ward number, admission date, and daily charges
- OutPatient — with appointment date and consultation fee

Analyze encapsulation requirements for sensitive data (diagnosis, prescription). Override a generateBill() method for each type and compare the billing logic differences. OOP Concepts: Encapsulation | Inheritance | Polymorphism

Your Answer

```
START
CLASS Patient
patientid,name,diagnosis,prescription
METHOD generatebill()
    PRINT "general patient bill"
END METHOD
END CLASS
CLASS inpatient extends patient
wardno,admissiondate,dailycharges
OVERRIDE METHOD generatebill()
bill=dailycharges*numberofdays
PRINT bill
END METHOD
END CLASS
CLASS outpatient extends patient
appointmentdate,consulationfee
OVERRIDE METHOD generatebill()
bill=conwsutataion fee
PRINT bill
END METHOD
END CLASS
STOP
```

 Save Answer

Q4 Shape Area Calculator using Polymorphism [BL3 – Apply]

10 marks

Create a Shape base class with an overridable calculateArea() method. Apply this to:

- Circle — area using radius
- Rectangle — area using length and breadth
- Triangle — area using base and height

Use method overriding to demonstrate runtime polymorphism. Call calculateArea() for each object.

OOP Concepts: Inheritance | Polymorphism | Method Overriding

Your Answer

```
create a base class shape
    define method calculateArea()
create class circle extends shape
    read radius
    override calculateArea()
    area=3.14*radius*radius
    display area

create class rectangle extends shape
    read length and breadth
    override calculatearea()
    area=length*breadth
    display area
create class triangle extends shape
    read base and height
    override calculatearea()
    area=0.5*base*height
    display area
MAIN
create shape object s1=circle
create shape object s2=rectangle
create shape object s3=triangle
call s1.calculatearea()
call s2.calculatearea()
call s3.calculatearea()
STOP
```

 Save Answer

Analyze and design a Product class hierarchy for an e-commerce system:

- Electronics — with warranty, brand, voltage
- Clothing — with size, fabric, gender
- Grocery — with expiryDate and weight

Override an applyDiscount() method for each category with different discount rules. Analyze why polymorphism makes the cart checkout process flexible and extensible. OOP Concepts: Polymorphism | Inheritance | Method Overriding

Your Answer

```
START
CLASS product
productid,name,price
METHOD applydiscount()
PRINT "general discount"
END METHOD
END CLASS
CLASS electronics extends product
warrenty,brand,voltage
OVERRIDE METHOD applydiscount()
discount=price*10/100
price=price-discount
END METHOD
END CLASS
CLASS clothing extends product
size,fabric,gender
OVERRIDE METHOD applydiscount()
discount=price*20/100
price=price-discount
END METHOD
END CLASS
STOP
```

 Save Answer

Q9 University Staff Hierarchy [BL4 – Analyze]

10 marks

Analyze the design of a Staff base class extended by TeachingStaff and NonTeachingStaff. Further extend TeachingStaff into Professor and Lecturer (multilevel inheritance). For each level:

- Identify which attributes should be private vs protected
- Override calculateAllowance() at each level
- Analyze how protected access enables inheritance without full exposure Justify the use of multilevel inheritance and explain how access modifiers enforce data hiding.

OOP Concepts: Multilevel Inheritance | Access Modifiers | Data Hiding

Your Answer

```
START
CLASS staff
PRIVATE id,name
PROTECTED salary
METHOD calculateallowance()
END METHOD
END CLASS
CLASS teachingstaff extends staff
OVERRIDE calculateallowance()
END METHOD
END CLASS
CLASS nonteachingstaff extends staff
OVERRIDE calculateallowance()
END CLASS

CLASS professor extends teachingstaff
OVERRIDE calculateallowance()
END METHOD
END CLASS
CLASS lecturer extends teachingstaff
OVERRIDE calculateallowance()
END METHOD
END CLASS
STOP
```

 Save Answer

Analyze and design a SmartDevice superclass with subclasses: SmartLight, SmartThermostat, and SmartLock. Override an executeCommand(cmd) method in each, where the same command (e.g., 'on', 'off', 'status') produces device-specific behavior. Analyze:

- How polymorphism allows a single controller loop to manage all devices
- Security implications of data hiding in SmartLock (PIN must never be exposed) Design the class hierarchy and explain the OOP principles applied at each level.

OOP Concepts: Polymorphism | Encapsulation | Inheritance

Your Answer

```
START
CLASS smartdevice
  devicename
  METHOD executecommand(cmd)
END METHOD
END CLASS
CLASS Smartlight extends smartdevice
  OVERRIDE executecommand(cmd)
  PERFORM light action
END METHOD
END CLASS
CLASS smartthermostat extends smartdevice
  OVERRIDE executecommand(cmd)
  PERFORM temperature action
END METHOD
END CLASS
CLASS smartlock extends smartdevice
  PRIVATE PIN
  OVERRIDE executecommand(cmd)
  IF PIN is valid
  PERFORM lock action
  END IF
END METHOD
END CLASS
STOP
```

 Save Answer